ZCAM LATAM Hub — Secure Azure Databricks Platform

Technical Design Proposal | Zurich System Administrator Assignment | Version 0.1 — Terraform
Status: Proposed design; not deployed or tested in an authorized Azure tenant.
Scope: Development and production Databricks workspaces, enterprise connectivity, private data access, secrets, cost controls and production resilience.

> **Implementation boundary.** The accompanying Terraform package implements: one reusable `modules/databricks` module, invoked by two independent root configurations (`environments/dev` and `environments/prod`). It configures a Databricks Premium workspace for each environment when run with valid Azure prerequisites. Network foundations, private endpoints, ADLS, Azure SQL, Key Vault, identity grants and disaster recovery remain architectural dependencies, not resources provisioned by this module.

1. Executive summary and assumptions

The solution extends the **existing** Hub VNet, Azure Firewall, VPN Gateway, Microsoft Entra ID tenant, Azure DevOps platform and Azure SQL Database instead of recreating them. Production and development already use separate subscriptions in the same Entra tenant. Each environment is proposed to have its own Azure Databricks Premium workspace with VNet-injected classic compute, Secure Cluster Connectivity, private access and least-privilege permissions. A warm recovery foundation is proposed for production.

Assumptions requiring ZCAM approval: (A1) The initial Azure region is Brazil South (`brazilsouth`); any secondary region and PII replication require Legal, Security and Data Governance approval. (A2) The 15 Data Engineers are represented by Entra ID groups. (A3) The two external HTTPS APIs are ingestion sources; no customer PII is sent to them. (A4) Azure SQL ownership and cross-subscription permissions are handled by its owning team. (A5) Compute schedules and data volumes follow the provisional cost model below. (A6) Classic VNet-injected compute is the initial model; serverless requires a separate network and cost assessment. (A7) The three named ADLS accounts are production data stores; development uses separate stores with synthetic or irreversibly de-identified data. (A8) Network ranges, resource names and the chosen Databricks private connectivity model must be validated before deployment.

2. Network design, isolation and data flow

2.1 Hub-and-Spoke and workspace access

The existing Hub retains the Azure Firewall and VPN Gateway connecting the São Paulo office. Development and production spokes are isolated in **separate subscriptions**, resource groups, VNets, Databricks workspaces, data stores, Key Vaults and access boundaries. Hub-to-spoke peering enables approved transit; there is no direct Dev-to-Prod route. Example non-overlapping address spaces are `10.20.0.0/16` (Prod) and `10.30.0.0/16` (Dev), subject to enterprise IPAM approval.

Each workspace requires **two existing dedicated subnets** delegated to `Microsoft.Databricks/workspaces`, associated with appropriate NSGs and compatible UDRs. The module expects `snet-adb-host-prod` / `snet-adb-container-prod` or the corresponding `-dev` names, derived from the `environment` input. In AzureRM, the `public_subnet_name` field is historical API terminology; the module sets `no_public_ip = true` so compute nodes are not assigned public IP addresses. Separate Private Endpoint subnets and approved agent connectivity are design prerequisites, not provisioned by v2.

The target is private workspace access: `public_network_access_enabled = false`, with the required Databricks front-end/back-end Private Link arrangements, regional browser authentication, private DNS and routing established **before** access is tested. Office users connect through the corporate VPN. A workspace with

public access disabled but missing Private Link or DNS may be unreachable. NSGs and UDRs must preserve required Databricks control-plane traffic. Approved Internet egress, including the two external API FQDNs on TCP 443, traverses the Hub Firewall; platform dependencies are allowlisted separately.

2.2 Data services and sensitive-data protection

The production data flow is:

> **External HTTPS APIs → approved ingestion via Hub Firewall → `stlatamraw` → Databricks classic compute → `stlatamcurated` (restricted PII) → `stlatamanalytics` (approved aggregates).**

Production also accesses the existing Azure SQL Database through an approved private endpoint and cross-subscription authorization. Development cannot reach production SQL or PII by default and uses independent development storage with safe test data.

All three production ADLS Gen2 accounts are designed with public network access disabled and `dfs`/`blob` private endpoints where required; private DNS uses `privatelink.dfs.core.windows.net` and `privatelink.blob.core.windows.net`. SQL uses `privatelink.database.windows.net`; Key Vault uses `privatelink.vaultcore.azure.net`. DNS, effective routes, storage/firewall restrictions and **negative connectivity tests** must demonstrate that customer PII never traverses the public Internet. The v2 Terraform workspace controls **do not, on their own, prove** this end-to-end requirement.

3. Identity, RBAC and data governance

Microsoft Entra ID is the workforce identity provider; MFA, Conditional Access and periodic access reviews follow enterprise policy. Grant access to groups rather than individual users. Proposed groups include `ZCAM-DBX-PROD-ADMINS` (restricted administration), `ZCAM-DBX-PROD-ENGINEERS` (production jobs/notebooks), `ZCAM-DBX-DEV-ENGINEERS` (development only), `ZCAM-DATA-PII-READERS` (approved PII access) and `ZCAM-PLATFORM-OPERATORS` (monitoring and incident response).

Apply RBAC at the smallest practical scope. Illustrative roles include Reader for authorized auditors; tightly scoped Resource Group Contributor for deployment identities where required; Network Contributor on the approved VNet for authorized network provisioning; Key Vault Secrets User for approved runtime identities; and Storage Blob Data Reader/Contributor at the necessary storage scope. Avoid broad Owner assignments, shared storage keys and persistent credentials. Azure resource RBAC is distinct from **Unity Catalog** data privileges: grant `USE CATALOG`, `USE SCHEMA`, `SELECT` and write permissions only where justified. Audit PII access; development groups receive no production PII privileges.

Azure DevOps should use branch protection, approvals, protected environments, private build agents with necessary connectivity and workload identity federation. A separate pipeline execution context and remote Terraform state must be maintained for each environment; the archive provides separate roots but does **not** configure remote backends or implement these pipelines.

4. Key Vault and secrets rotation

Use separate production and development Key Vaults with Azure RBAC, Private Endpoints, private DNS, public access restrictions, soft delete, purge protection, diagnostic logs and least privilege. Store the two external API credentials and any legacy secrets that cannot be replaced by identity-based connections. Prefer managed identities and short-lived credentials for ADLS and Azure SQL where supported.

Secrets must not be embedded in notebooks, Git, Terraform variable files, pipeline YAML, logs or outputs. An authorized VNet-connected ingestion workload retrieves API credentials from a private vault at runtime and makes the required HTTPS calls through the Firewall. Assign ownership, track expiration, alert before expiry, rotate with overlap where supported and maintain an operational runbook. A **90-day maximum review/rotation window** is proposed as an initial control, subject to each provider's security policy and capability.

Compatibility trade-off: This design does not presume that Azure Key Vault-backed Databricks secret scopes will work with a strict RBAC-only and private-endpoint-only vault posture. Scope permissions and network requirements must be validated separately; a VNet-connected identity-based consumer is the proposed baseline. Any exception requires security review.

5. Terraform: exact implementation alignment

The submitted IaC chooses **Databricks workspace with VNet injection**. It contains **one child module**, reused by two root configurations; it does **not** contain two independently developed Databricks modules.

Terraform file map:

- **Shared module — `modules/databricks/main.tf`:** defines `azurerm_databricks_workspace.this` and its child-module `workspace_id` output.
- **Production root — `environments/prod/main.tf`:** selects the Prod `subscription_id` and calls the shared module with `environment = "prod"`.
- **Development root — `environments/dev/main.tf`:** selects the Dev `subscription_id` and calls the same module with `environment = "dev"`.
- **Environment inputs — `terraform.tfvars.example` in each environment folder:** placeholder Azure resource IDs, not deployment-ready configuration.
- **Supporting files — `.gitignore` and `README.md`:** local-state/secret exclusions and operating instructions.

Workspace configurations encoded in the module: Azure Databricks **Premium**; workspace names `adb-zcam-prod-brs` / `adb-zcam-dev-brs`; managed Resource Groups `rg-adb-managed-zcam-prod-brs` / `rg-adb-managed-zcam-dev-brs`; VNet injection into the corresponding existing host/container subnets; `no_public_ip = true`; `public_network_access_enabled = false`; `network_security_group_rules_required = "NoAzureDatabricksRules"`; `default_storage_firewall_enabled = true`; an existing Access Connector ID; and environment tags. The managed-storage firewall setting applies to **Databricks-managed storage**, not the three separate ADLS Gen2 accounts in the architecture.

The roots accept `subscription_id`, `resource_group_name`, `vnet_id`, `access_connector_id` and v2al `location` (default `brazilsouth`). The workspace/subnet naming convention includes a fixed `brs` suffix and should be reviewed if the region changes. The shared module exports `workspace_id`; the current root configurations do not re-export that output. The code references existing VNet, delegated subnets, NSGs and Access Connector; it does **not** create them or Databricks Private Link endpoints, DNS, SQL connectivity, Key Vault, users, ADLS storage, monitoring or DR resources.

State and execution: The two environment roots are intended for **different subscriptions and independent Terraform states**. The archive does not configure a remote backend; enterprise deployment requires separate secured Azure Storage state backends, RBAC, locking and deployment approvals. Use `terraform -chdir=environments/prod ...` and `terraform -chdir=environments/dev ...` for independent `init`, `fmt`, `validate` and authorized `plan`. Real plans require valid Azure subscriptions and pre-existing resources. The package has **not been deployed**, and no successful cloud plan or integration test is claimed.

6. Monthly cost estimate (USD, illustrative)

This is an **allowance-based planning scenario, not an Azure price quotation**. DBUs, VM SKUs, region, network processing, data volumes and enterprise discounts require validation against the Azure Pricing Calculator and the organization's agreement.

Service / workload	Basis	USD/month
Production Databricks compute + DBUs	8 h/day × 22 days × $16 blended/h	2,816
Development Databricks compute + DBUs	4 h/day × 20 days × $8 blended/h	640
Production and separate Dev storage accounts	Capacity and transactions allowance	600
Production DR replication, backups and preparation	Secondary-site allowance	800
Private networking, DNS, allocated shared Firewall	Incremental / allocated	900
Azure Monitor, logs and alerting	Controlled retention	300
Key Vault	Secret and operation allowance	50
API and data transfer	Governed by job schedules and egress	200
Subtotal		**6,306**
Contingency (~20%)		**1,261**
Illustrative combined total	**Target: below USD 8,000**	**7,567**

The model excludes historical Hub fixed costs and the existing Azure SQL baseline, except for the allocated network amount shown. If the cost ceiling must include those services, or if actual usage exceeds the allowances, rebaseline before approval. Apply environment tags, Azure budget alerts (50/75/90/100%), compute policies, automatic cluster termination, scheduled Dev jobs and monthly FinOps reviews. The **USD 7,567** estimate is conditional, not guaranteed or enforced by Terraform.

7. Production disaster recovery: RPO 4h / RTO 2h

RPO 4 hours and RTO 2 hours are unverified design objectives, not measured outcomes. Propose a warm, security-approved secondary-region foundation with private networking, DNS, deployable workspace configuration, required access groups, job definitions, approved storage replication and dependency recovery. Data residency and PII replication require explicit approval. Secondary-environment provisioning and failover automation are **outside the Terraform module**.

Aim for recovery-consistent Delta/data checkpoints materially more frequent than four hours (for example, every two hours); record the last valid snapshot and monitor replication lag. Replicate required Unity Catalog metadata and deployable assets, and validate end-to-end recovery consistency. The existing Azure SQL database is a critical dependency: its owner must approve geo-recovery/failover or an explicit degraded mode. A restored Databricks workspace without accessible SQL may not meet the business RTO.

Timed DR test: declare the incident; stop or fence primary writes; verify recovery state; activate approved private DNS/routing and secondary compute; restore job/configuration and validate Unity Catalog, SQL and Key Vault access; run data reconciliation and smoke tests; then resume workloads. Measure both elapsed time to usable service and age of the latest usable data. Rehearse at least quarterly and document failback procedures.

8. Trade-offs, validation and acceptance

1. **Private Link by default:** minimizes unintended public exposure but increases DNS, endpoint, routing and operational complexity; test browser authentication and back-end connectivity before cutover.

2. **Classic VNet-injected compute:** supports explicit NSG/Firewall controls; serverless requires separate connectivity and cost assessment.
3. **Separate Dev and Prod roots/subscriptions:** strengthens isolation and reduces blast radius; it also requires disciplined state separation and independent permissions.
4. **Private Key Vault with identity-based consumption:** favors network isolation and RBAC over convenience integrations requiring exceptions.
5. **Warm DR rather than active-active:** reduces idle cost, but the two-hour RTO depends on successful automation and drills.
6. **Reuse of existing Hub services:** limits new infrastructure, while adding dependency on central Firewall, VPN, DNS and change control.

Acceptance checks: Terraform formatting and static validation for **both** roots; authorized per-environment `plan`; subnet delegation/NSG/UDR review; functional workspace Private Link and DNS; negative tests proving no public path to PII or SQL; explicit approval of the two HTTPS API destinations; strict Dev isolation; RBAC and Unity Catalog access tests; Key Vault rotation/alert test; remote state and pipeline review; revised service-level pricing; and timed RPO/RTO evidence. None of these deployment or integration checks is represented as completed by the archive.

Implementation status by deliverable

Deliverable	What it demonstrates	Status
Architecture diagram	Hub-and-Spoke, DEV/PROD, data flow, PEs, Vault and DR	Proposed design
Technical design (this document)	Decisions, RBAC, Key Vault, costs, recovery and trade-offs	Proposed design
Terraform	**One reusable Databricks module** invoked separately from Dev and Prod roots	Code example; no cloud deployment demonstrated

Official reference documentation

Azure Databricks: VNet injection · Private Link · Secret scopes · Disaster recovery

Azure data services: Storage Private Endpoints · Azure SQL Private Link

Terraform: AzureRM `azurerm_databricks_workspace`